OAVS · TGT COMPUTER TEACHER EXAMINATION

MCQ Question Bank

Java Object-Oriented Programming — Core Concepts

CHAPTER / TOPIC

Objects and Classes

210	16	12
MCQS	SECTIONS	SYLLABUS TOPICS

Volume 1 · Prepared for OAVS TGT Computer Science Aspirants

Table of Contents

SEC 01	Object Fundamentals	16 Qs
SEC 02	Pass by Value	14 Qs
SEC 03	'this' Reference	14 Qs
SEC 04	Data Hiding and Encapsulation	12 Qs
SEC 05	Overloading	15 Qs
SEC 06	Overriding	15 Qs
SEC 07	Constructors	15 Qs
SEC 08	Finalization	9 Qs
SEC 09	Subclasses (Inheritance)	14 Qs
SEC 10	Superclass and Subclass Object Relationship	11 Qs
SEC 11	Implicit Subclass-to-Superclass Conversion	11 Qs
SEC 12	Dynamic Method Dispatch	14 Qs
SEC 13	Comparison Questions	12 Qs
SEC 14	Code and Output-Based Questions	14 Qs
SEC 15	Difficult and Tricky Questions	12 Qs
SEC 16	Mixed OAVS TGT Practice Set	12 Qs

Section 1 — Object Fundamentals

16 Questions

Q1 | Object Fundamentals DEFINITION

What is an object in Java?

A. An instance of a class having state and behavior
B. A primitive data type
C. A keyword used to define a class
D. A blueprint used to create classes

Ans: A

Explanation: An object is a runtime instance of a class; it possesses state (fields) and behavior (methods).

Q2 | Object Fundamentals CONCEPT

Which of the following best describes the 'state' of an object?

A. The class from which it was created
B. The memory address of the object
C. The set of methods it can execute
D. The values currently stored in its instance variables

Ans: D

Explanation: State refers to the data (values of instance variables) an object holds at a given time.

Q3 | Object Fundamentals CONCEPT

The 'behavior' of an object is defined by its:

A. Class name
B. Methods
C. Constructors only
D. Instance variables

Ans: B

Explanation: Behavior is what an object can do, implemented through its methods.

Q4 | Object Fundamentals CONCEPT

An object is created in Java using which operator?

A. this
B. new
C. create
D. class

Ans: B

Explanation: The new operator allocates memory and creates a new instance of a class.

Q5 | Object Fundamentals CONCEPT

Two references pointing to the same object in memory are said to have the same:

A. Behavior
B. State
C. Class hierarchy
D. Identity

Ans: D

Explanation: Object identity refers to the unique existence of an object in memory.

Q6 | Object Fundamentals CONCEPTUAL

If two different objects have exactly the same field values, are they identical?

A. Yes, because Java treats equal state as equal identity
B. No, they belong to different classes
C. Yes, they are always the same object
D. No, they have the same state but different identities

Ans: D

Explanation: Equal state does not imply the same identity; each object created with new has a distinct identity.

Q7 | Object Fundamentals FILL IN THE BLANK

A class acts as a _____ for creating objects.

A. Method
B. Template/blueprint
C. Subclass
D. Constructor call

Ans: B

Explanation: A class defines the structure and behavior that its objects will have; it is the blueprint for object creation.

Q8 | Object Fundamentals FILL IN THE BLANK

Every object in Java is an instance of exactly one _____ at the time of creation.

A. package
B. method
C. interface
D. class

Ans: D

Explanation: An object is instantiated from a specific class using new.

Q9 | Object Fundamentals IDENTIFICATION

Which statement about objects is TRUE?

A. An object exists only during compilation
B. Objects and classes are the same thing
C. An object cannot have more than one instance variable
D. An object combines data and the methods that operate on that data

Ans: D

Explanation: Objects bundle state (data) with behavior (methods), the core idea of object-based design.

Q10 | Object Fundamentals CONCEPTUAL

What happens when an object is created without ever being assigned to a reference variable?

A. Java throws a compile-time error
B. It exists in memory but becomes unreachable and inaccessible
C. It is stored as a static object
D. It automatically becomes a class

Ans: B

Explanation: An object created without a reference has no way to be accessed and is effectively unreachable.

Q11 | Object Fundamentals IDENTIFICATION

Which of these is NOT a characteristic of an object?

A. It occupies no memory until a method is called
B. It has state
C. It has identity
D. It has behavior

Ans: A

Explanation: An object occupies memory as soon as it is created, not only when a method is invoked.

Q12 | Object Fundamentals CONCEPT

A reference variable in Java stores:

A. Only the class name of the object
B. The memory reference to an object
C. The object's methods as text
D. The complete object copy

Ans: B

Explanation: Reference variables hold a reference (address) to an object stored in memory, not the object itself.

Q13 | Object Fundamentals CONCEPT

Multiple reference variables can refer to:

A. The same single object
B. Only their own separate object
C. A class, but never an object
D. Nothing at runtime

Ans: A

Explanation: In Java, several references can point to the same object simultaneously.

Q14 | Object Fundamentals SCENARIO

Object A and Object B are instances of the same class Car. Changing a field of A does NOT affect B because:

A. Each object maintains its own independent copy of instance variables
B. B was never initialized
C. Cars cannot have instance variables
D. Objects of the same class always share fields

Ans: A

Explanation: Instance variables belong to each object individually, so changes to one object's state don't affect another.

Q15 | Object Fundamentals CONCEPTUAL

The relationship between a class and its objects is best described as:

A. Classes and objects are interchangeable terms
B. One class can be used to create many objects
C. One object can belong to many unrelated classes
D. A class can only ever create one object

Ans: B

Explanation: A single class definition can be used to instantiate multiple independent objects.

Q16 | Object Fundamentals ASSERTION-REASON

Assertion (A): All objects of a class share the same values for their instance variables. Reason (R): Instance variables are allocated separately for every object.

A. Both A and R are false

B. A is false, R is true

C. A is true, R is false

D. Both A and R are true, R explains A

Ans: B

Explanation: Instance variables are allocated per-object, so objects generally hold different values; the assertion is false while the reason is true.

Section 2 — Pass by Value

14 Questions

Q17 | Pass by Value CONCEPT

Java passes arguments to methods using which mechanism?

A. Pass by value
B. Pass by reference
C. Pass by pointer
D. Pass by name

Ans: A

Explanation: Java always passes a copy of the argument's value to a method, regardless of the type.

Q18 | Pass by Value CONCEPTUAL

When a primitive int is passed to a method and modified inside it, what happens to the original variable?

A. It remains unchanged
B. It is updated to the new value
C. A compile-time error occurs
D. It becomes undefined

Ans: A

Explanation: Since a copy of the value is passed, changes inside the method do not affect the caller's original variable.

Q19 | Pass by Value DEFINITION

The variables listed in a method's declaration (e.g., void m(int x)) are called:

A. Actual parameters
B. Static parameters
C. Return parameters
D. Formal parameters

Ans: D

Explanation: Formal parameters are the variables defined in the method signature that receive values.

Q20 | Pass by Value DEFINITION

The values supplied when calling a method (e.g., m(5)) are called:

A. Actual parameters
B. Instance parameters
C. Formal parameters
D. Reference parameters

Ans: A

Explanation: Actual parameters (arguments) are the real values passed to a method at the call site.

Q21 | Pass by Value CONCEPTUAL

When an object reference is passed to a method, what exactly is copied?

A. Nothing is copied
B. The entire object including all its fields
C. A copy of the reference (the memory address), not the object itself
D. Only the object's class name

Ans: C

Explanation: Java copies the reference value; both the original and copied reference point to the same object.

Q22 | Pass by Value CONCEPTUAL

If a method changes a field of an object through a passed reference, is this change visible outside the method?

A. Yes, but only for final objects
B. No, references are always copies of copies
C. Yes, because both references point to the same object
D. No, because objects are passed by reference in Java

Ans: C

Explanation: Since the copied reference points to the same object, modifying the object's fields affects the shared object.

Q23 | Pass by Value CONCEPTUAL

If inside a method a passed reference is reassigned to a brand-new object, does the caller's original reference change?

A. Yes, the caller's reference now points to the new object
B. Yes, but only for arrays
C. No, only the local copy of the reference is reassigned
D. A compile-time error occurs

Ans: C

Explanation: Reassigning the local reference variable inside the method only affects that copy, not the caller's reference.

Q24 | Pass by Value MISCONCEPTION

A student says 'Java passes objects by reference.' Why is this technically incorrect?

A. Java always passes a copy of the reference value, which is still pass by value
B. Objects cannot be passed as parameters
C. It is actually correct terminology
D. Java never allows object references to be passed

Ans: A

Explanation: Java uses pass by value uniformly; for objects, the value passed happens to be a reference, not the object.

Q25 | Pass by Value CODE/OUTPUT

```
void change(int x){ x = x + 10; } ... int a = 5; change(a); System.out.println(a); What is the output?
```

A. 5
B. 10
C. Compilation error
D. 15

Ans: A

Explanation: Since int is primitive, a copy of a is passed; modifying x inside change() has no effect on a.

Q26 | Pass by Value CODE/OUTPUT

```
class Box{int v;} void update(Box b){ b.v = 100; } Box obj = new Box(); obj.v = 5; update(obj);
print(obj.v); What is the output?
```

A. 5
B. 0
C. Compilation error
D. 100

Ans: D

Explanation: The copied reference inside update() points to the same Box object, so modifying its field changes the shared object.

Q27 | Pass by Value CODE/OUTPUT

```
void reset(Box b){ b = new Box(); b.v = 999; } Box obj = new Box(); obj.v = 1; reset(obj); print(obj.v);
What is the output?
```

A. 1
B. Compilation error
C. 0
D. 999

Ans: A

Explanation: reset() reassigns its local copy b to a new object; obj still refers to the original object, so obj.v remains 1.

Q28 | Pass by Value CONCEPTUAL

Pass by value in Java ensures that:

A. Method parameters are always final
B. Arrays are passed differently from other objects
C. A method can always change the caller's primitive variables
D. A method cannot change the caller's original reference variable, only the object it points to (for objects)

Ans: D

Explanation: The key guarantee is that the local variable in the method is independent of the caller's variable, though shared object contents can still be mutated.

Q29 | Pass by Value CONCEPTUAL

Which of these is passed by value exactly like a primitive, in terms of the reference itself?

A. An object reference
B. The heap memory block
C. The object
D. The class definition

Ans: A

Explanation: The reference variable itself is copied by value; what it points to (the object) is shared, not copied.

Q30 | Pass by Value ASSERTION-REASON

Assertion (A): Swapping two int variables using a swap(int a, int b) method works in Java. Reason (R): Java passes primitives by value.

A. Both are false
B. A is true, R is false
C. Both A and R are true, R explains A
D. A is false, R is true

Ans: D

Explanation: Because primitives are passed by value, swap() only affects local copies, so the swap does not persist outside the method.

Section 3 — 'this' Reference

14 Questions

Q31 | 'this' Reference DEFINITION

What does the 'this' keyword refer to?

A. The current object on which a method or constructor is invoked

B. A static reference shared by all objects

C. The superclass of the current object

D. The name of the current class

Ans: A

Explanation: this always refers to the object whose method or constructor is currently executing.

Q32 | 'this' Reference CONCEPT

Which of the following is a common use of 'this'?

A. Distinguishing an instance variable from a parameter with the same name

B. Creating a static method

C. Importing a package

D. Declaring a new class

Ans: A

Explanation: When a parameter shadows an instance variable, this.name explicitly refers to the instance variable.

Q33 | 'this' Reference CODE/OUTPUT

```
class Student{ String name; Student(String name){ this.name = name; } } Here, this.name refers to:
```

A. A static variable

B. A local variable inside a loop

C. The instance variable name of the object

D. The parameter name

Ans: C

Explanation: this.name accesses the object's field, avoiding ambiguity with the constructor parameter of the same name.

Q34 | 'this' Reference ERROR-IDENTIFICATION

```
class Student{ String name; Student(String name){ name = name; } } What is the likely issue with this constructor?
```

A. It sets name to null explicitly

B. It causes a compilation error

C. It correctly initializes the instance variable

D. It assigns the parameter to itself; the instance variable name remains unset

Ans: D

Explanation: Without this, 'name = name' refers to the parameter on both sides, so the instance field is never assigned.

Q35 | 'this' Reference RULE-BASED

Can 'this' be used inside a static method?

A. Yes, always

B. No, because static methods are not associated with any particular object

C. Yes, if the class has no constructors

D. Yes, but only in the main method

Ans: B

Explanation: this refers to an object instance, but static methods belong to the class, not to any object.

Q36 | 'this' Reference DEFINITION

One constructor calling another constructor of the same class using 'this(...)' is called:

A. Constructor finalization
B. Constructor hiding
C. Constructor overriding
D. Constructor chaining

Ans: D

Explanation: this(...) invokes another constructor in the same class, allowing shared initialization logic.

Q37 | 'this' Reference CONCEPT

In a method, 'this' allows an object to:

A. Directly modify the class definition
B. Skip calling its own constructor
C. Access another unrelated object's private data
D. Pass a reference to itself to another method

Ans: D

Explanation: this can be passed as an argument, letting other code reference back to the current object.

Q38 | 'this' Reference MISCONCEPTION

A beginner writes 'int x = this.x + this.x;' expecting it to double a local variable x. What's wrong?

A. The code correctly doubles the local variable
B. this.x is invalid syntax
C. this can only be used in constructors
D. this.x refers to the instance variable, not the local variable x

Ans: D

Explanation: this.x always refers to the object's field; a plain local variable x is unrelated and unaffected.

Q39 | 'this' Reference CODE/OUTPUT

```
class P{ int x=5; void show(){ int x=10; System.out.println(this.x); } } What is the output of show()?
```

A. 10
B. 5
C. Compilation error
D. 15

Ans: B

Explanation: this.x accesses the instance variable (5), ignoring the local variable x (10) which shadows it.

Q40 | 'this' Reference IDENTIFICATION

Which statement is FALSE about 'this'?

A. 'this' refers to the current object
B. 'this' can be passed as a method argument
C. 'this' can be used to refer to a static variable exclusively
D. 'this' can resolve naming conflicts

Ans: C

Explanation: this relates to instance context; it is not used exclusively for static members, which belong to the class.

Q41 | 'this' Reference CONCEPT

'this' is typically used in a constructor to:

A. Declare a new object

B. Assign parameter values to instance variables with matching names

C. Convert the object to its superclass

D. Define the return type of the constructor

Ans: B

Explanation: Constructors often use this.field = parameter to initialize instance variables when parameter names match field names.

Q42 | 'this' Reference FILL IN THE BLANK

Inside an instance method, _____ always refers to the object through which the method was invoked.

A. self

B. static

C. super

D. this

Ans: D

Explanation: this is Java's keyword for referring to the invoking object within instance context.

Q43 | 'this' Reference SCENARIO

Method compareTo(Person other) needs to compare the current object's age with another Person's age. Which expression correctly refers to the current object's age?

A. this.age

B. Person.age

C. other.age only

D. super.age

Ans: A

Explanation: this.age explicitly accesses the age field of the object on which compareTo was called.

Q44 | 'this' Reference ASSERTION-REASON

Assertion (A): 'this' can be used inside every method of a class. Reason (R): 'this' refers to the object currently executing the method.

A. A is false, R is true

B. A is true, R is false

C. Both are false

D. Both A and R are true, R explains A

Ans: A

Explanation: this cannot be used in static methods since there is no invoking object, so the assertion is false even though the reason is generally true.

Section 4 — Data Hiding and Encapsulation

12 Questions

Q45 | Data Hiding and Encapsulation DEFINITION

Data hiding refers to:

A. Restricting direct access to an object's internal data
B. Deleting unused variables from a class
C. Hiding entire classes from a package
D. Compiling code without showing errors

Ans: A

Explanation: Data hiding conceals the internal state of an object from outside interference and misuse.

Q46 | Data Hiding and Encapsulation DEFINITION

Encapsulation is best described as:

A. Converting object data into static fields
B. Removing constructors from a class
C. Bundling data and the methods that operate on it within a single unit, while restricting direct access
D. Separating class definitions from method definitions

Ans: C

Explanation: Encapsulation wraps state and behavior together and controls access via methods, protecting the object's data.

Q47 | Data Hiding and Encapsulation CONCEPT

What is a primary benefit of data hiding?

A. It protects the internal state of an object from unintended external modification
B. It increases the number of classes needed
C. It removes the need for constructors
D. It eliminates the need for methods

Ans: A

Explanation: By restricting direct access, data hiding prevents external code from corrupting an object's internal data.

Q48 | Data Hiding and Encapsulation CONCEPT

Controlled access to an object's data is typically provided through:

A. Removing all methods
B. Public methods (such as getters and setters) that mediate access to private fields
C. Making all fields public
D. Using only static fields

Ans: B

Explanation: Encapsulation exposes a controlled interface (methods) while keeping the underlying fields hidden.

Q49 | Data Hiding and Encapsulation COMPARISON

How are data hiding and encapsulation related?

A. Data hiding is a form of inheritance
B. Data hiding is a key outcome achieved through encapsulation
C. Encapsulation always requires overriding
D. They are entirely unrelated concepts

Ans: B

Explanation: Encapsulation is the technique of bundling data with methods, and data hiding is the protective result of restricting access.

Q50 | Data Hiding and Encapsulation IDENTIFICATION

Which scenario best illustrates encapsulation?

A. A subclass overrides all superclass methods

B. A class exposes methods to get and set a field's value instead of allowing direct field access

C. Two unrelated classes share the same variable name

D. A class has no methods at all

Ans: B

Explanation: Encapsulation typically manifests through controlled access methods rather than exposing fields directly.

Q51 | Data Hiding and Encapsulation IDENTIFICATION

Which of these design choices supports data hiding most directly?

A. Avoiding the use of methods

B. Declaring instance variables as public

C. Declaring instance variables as private

D. Making the class itself private in the same file

Ans: C

Explanation: Marking fields private prevents direct outside access, enforcing data hiding.

Q52 | Data Hiding and Encapsulation COMPARISON

How does encapsulation differ conceptually from mere data hiding?

A. Encapsulation only applies to constructors

B. Encapsulation includes bundling both data and behavior, while data hiding specifically concerns restricting access to data

C. They are exact synonyms with no distinction

D. Data hiding is broader and includes method behavior too

Ans: B

Explanation: Encapsulation is the broader design principle of combining state and behavior; data hiding emphasizes concealing state.

Q53 | Data Hiding and Encapsulation CONCEPTUAL

What could go wrong if an object's fields are NOT protected through data hiding?

A. The object would lose its identity

B. The class would fail to compile

C. External code could directly set the fields to invalid or inconsistent values

D. The object would automatically become static

Ans: C

Explanation: Without protection, outside code can freely modify fields, potentially violating the object's intended constraints.

Q54 | Data Hiding and Encapsulation CONCEPTUAL

Encapsulation supports which broader software quality?

A. Automatic thread synchronization

B. Faster compilation of unrelated classes

C. Maintainability, since internal implementation can change without affecting external code

D. Guaranteed absence of runtime errors

Ans: C

Explanation: Because internal details are hidden behind a stable interface, implementation can evolve without breaking external code.

Q55 | Data Hiding and Encapsulation FILL IN THE BLANK

A class whose fields are private and accessed only through public methods is said to exhibit strong _____.

A. encapsulation
B. overriding
C. inheritance
D. finalization

Ans: A

Explanation: This design pattern is the hallmark of well-encapsulated classes.

Q56 | Data Hiding and Encapsulation SCENARIO

A BankAccount class keeps 'balance' private and only allows changes through a deposit() method that validates the amount. This primarily demonstrates:

A. Dynamic method dispatch
B. Method overriding
C. Constructor overloading
D. Data hiding combined with encapsulation

Ans: D

Explanation: Restricting direct access to balance and mediating changes through a validating method is a classic encapsulation example.

Section 5 — Overloading

15 Questions

Q57 | Overloading DEFINITION

Method overloading means:

A. Defining a method in a subclass with the same signature as in the superclass

B. Reusing a variable name across multiple classes

C. Declaring a method as final

D. Defining multiple methods in the same class with the same name but different parameter lists

Ans: D

Explanation: Overloading allows methods sharing a name to be distinguished by differing parameter lists within the same class.

Q58 | Overloading RULE-BASED

Which of the following can distinguish overloaded methods?

A. The method body content

B. The number or types of parameters

C. The return type alone

D. The access modifier alone

Ans: B

Explanation: Overload resolution is based on the parameter list, not on return type or access modifier alone.

Q59 | Overloading RULE-BASED

void add(int a, int b) and int add(int a, int b) - is this valid overloading?

A. No, because only the return type differs; the parameter list is identical

B. Yes, because the return types differ

C. Yes, because one method returns void

D. No, because overloading is never allowed with void

Ans: A

Explanation: Overloading requires a different parameter list; differing only in return type causes a compile-time error.

Q60 | Overloading IDENTIFICATION

Which of these method pairs represents VALID overloading?

A. void show(int a) and void show(int b)

B. void show(int a) and void display(int a)

C. void show(int a) and void show(int a, int b)

D. void show(int a) and int show(int a)

Ans: C

Explanation: Differing in the number of parameters (one vs two) is a valid basis for overloading.

Q61 | Overloading CONCEPTUAL

Overloading primarily helps achieve:

A. Compile-time (static) polymorphism

B. Object finalization

C. Runtime (dynamic) polymorphism

D. Data hiding

Ans: A

Explanation: Since the compiler resolves which overloaded method to call based on argument types at compile time, overloading is static polymorphism.

Q62 | Overloading IDENTIFICATION

Given print(int x), print(double x), and print(String x), which one is called for print(5)?

A. print(double x)
B. print(int x)
C. print(String x)
D. Compilation error due to ambiguity

Ans: B

Explanation: The literal 5 is an int, so the exact-match overload print(int x) is selected.

Q63 | Overloading IDENTIFICATION

Given print(int x) and print(double x), which is called for print(5.0)?

A. print(int x)
B. print(double x)
C. Compilation error
D. Both are called

Ans: B

Explanation: 5.0 is a double literal, matching print(double x) directly.

Q64 | Overloading RULE-BASED

Can overloaded methods have different access modifiers?

A. No, only constructors can vary access modifiers
B. Yes, access modifiers can differ between overloaded methods
C. Yes, but only if parameter types are identical
D. No, all overloaded methods must share the same access modifier

Ans: B

Explanation: Access modifiers are independent of overload resolution rules and can differ freely among overloaded methods.

Q65 | Overloading MISCONCEPTION

A student claims that changing only a parameter's name (not its type) creates a valid overload. Is this correct?

A. No, because overloading requires different method names
B. Yes, parameter names are sufficient to overload a method
C. No, parameter names do not affect overload resolution; only type/number/order matter
D. Yes, but only for private methods

Ans: C

Explanation: Overloading depends solely on the parameter list's types, count, or order - not on parameter names.

Q66 | Overloading CODE/OUTPUT

```
int calc(int a, int b){return a+b;} double calc(double a, double b){return a+b;} calc(2,3) invokes which version?
```

A. Compilation error due to ambiguity
B. Both methods execute
C. calc(int, int)
D. calc(double, double)

Ans: C

Explanation: Since 2 and 3 are int literals, they match calc(int,int) exactly, so that overload executes.

Q67 | Overloading RULE-BASED

Which of the following is TRUE about overloading and parameter order?

A. Parameter order never affects overloading

B. Changing the order of different-typed parameters (e.g., (int,double) vs (double,int)) can create valid overloads

C. Overloading requires identical parameter order across all versions

D. Order only matters for constructors, not methods

Ans: B

Explanation: A different order of differing parameter types produces a distinct signature, enabling valid overloading.

Q68 | Overloading SCENARIO

A Calculator class needs an add() method that can accept either two ints or two doubles. Which technique should be used?

A. Object finalization

B. Dynamic method dispatch

C. Method overriding

D. Method overloading

Ans: D

Explanation: Providing multiple add() methods with different parameter types within the same class is a textbook overloading scenario.

Q69 | Overloading FILL IN THE BLANK

Overloaded methods must have the _____ name but a different parameter list.

A. same

B. static

C. final

D. different

Ans: A

Explanation: The defining feature of overloading is the shared method name paired with differing parameter signatures.

Q70 | Overloading CODE/OUTPUT

```
void greet(){print('Hi');} void greet(String name){print('Hi '+name);} Calling greet('Sam') outputs:
```

A. Hi

B. Hi null

C. Compilation error

D. Hi Sam

Ans: D

Explanation: The call greet('Sam') matches the overloaded version accepting a String parameter.

Q71 | Overloading ASSERTION-REASON

Assertion (A): Two methods differing only in their thrown-exception clause, with an identical parameter list, are validly overloaded. Reason (R): Overload resolution depends only on the parameter list.

A. Both are false

B. A is true, R is false

C. A is false, R is true

D. Both A and R are true, R explains A

Ans: C

Explanation: Since exceptions are not part of the parameter list, two methods with an identical parameter list cannot be overloaded on that basis alone; the assertion is false.

Section 6 — Overriding

15 Questions

Q72 | Overriding DEFINITION

Method overriding occurs when:

A. A class defines two methods with the same name but different parameters

B. A method is declared without a body

C. A subclass provides its own implementation of a method already defined in its superclass with the same signature

D. A constructor is called more than once

Ans: C

Explanation: Overriding redefines inherited method behavior in a subclass using an identical method signature.

Q73 | Overriding CONCEPTUAL

Overriding is primarily used to achieve:

A. Data hiding

B. Compile-time polymorphism

C. Runtime (dynamic) polymorphism

D. Constructor chaining

Ans: C

Explanation: Because the actual method executed is determined at runtime based on the object's real type, overriding enables dynamic polymorphism.

Q74 | Overriding RULE-BASED

For a subclass method to override a superclass method, the method signature must be:

A. Completely different

B. Different only in return type

C. Different only in parameter names

D. The same as the superclass method

Ans: D

Explanation: Overriding requires an identical method name and parameter list as the superclass version.

Q75 | Overriding IDENTIFICATION

```
class Animal{void sound(){print('generic');}} class Dog extends Animal{void sound(){print('bark');}} Dog's sound() is an example of:
```

A. Constructor overloading

B. Method overloading

C. Method overriding

D. Data hiding

Ans: C

Explanation: Dog redefines the inherited sound() method with the same signature, which is overriding.

Q76 | Overriding COMPARISON

Which of the following distinguishes overriding from overloading?

A. There is no meaningful difference between them

B. Overriding is resolved at compile time; overloading at runtime

C. Overriding involves a superclass and subclass relationship; overloading occurs within the same class

D. Overriding always changes the parameter list; overloading never does

Ans: C

Explanation: Overriding is inherently tied to inheritance, while overloading happens among methods in the same class.

Q77 | Overriding RULE-BASED

Which relationship must exist between the classes for overriding to occur?

A. Superclass and subclass (inheritance) relationship
B. The classes must be in different packages only
C. No particular relationship is required
D. The classes must be unrelated

Ans: A

Explanation: Overriding specifically involves a subclass redefining a method inherited from its superclass.

Q78 | Overriding CODE/OUTPUT

```
class A{void show(){print('A');}} class B extends A{void show(){print('B');}} A obj = new B(); obj.show();
Output?
```

A. B
B. AB
C. A
D. Compilation error

Ans: A

Explanation: Even though the reference type is A, the actual object is B, so the overridden show() in B executes.

Q79 | Overriding MISCONCEPTION

A student overrides a method but accidentally changes its parameter list. What actually happens?

A. The program fails to compile in all cases
B. It becomes a constructor
C. Java automatically corrects the parameter list
D. It becomes an overloaded method instead of an overriding one

Ans: D

Explanation: Changing the parameter list breaks the identical-signature requirement, so it is treated as a new overloaded method.

Q80 | Overriding RULE-BASED

Which of these is a rule directly relevant to overriding in this syllabus context?

A. The overriding method must be defined before the superclass method
B. The overriding method must always be declared static
C. The overriding method must have the same name and parameter list as the superclass method
D. The overriding method must be defined in the same class

Ans: C

Explanation: Matching the method signature exactly is the essential requirement to override an inherited method.

Q81 | Overriding IDENTIFICATION

A Vehicle class has method start(); a Car subclass also defines start() with the same signature. Calling start() on a Car object executes:

A. Vehicle's version of start()
B. Both versions in sequence
C. Car's version of start()
D. Neither version

Ans: C

Explanation: The subclass's overriding implementation takes precedence when invoked on a subclass instance.

Q82 | Overriding FILL IN THE BLANK

Overriding allows a subclass to _____ behavior inherited from its superclass.

A. rename
B. redefine
C. duplicate
D. delete

Ans: B

Explanation: Overriding replaces (redefines) the inherited method's implementation in the subclass.

Q83 | Overriding SCENARIO

A Shape class has a method area() that returns 0. Circle and Square subclasses each provide their own area() calculation. This design uses:

A. Method overloading
B. Pass by value
C. Method overriding
D. Data hiding

Ans: C

Explanation: Each subclass supplies its own specific implementation of the inherited area() method, which is overriding.

Q84 | Overriding STATEMENT-BASED

Which statement is correct: 'Overriding can occur without inheritance.'

A. False, but only for constructors
B. True, overriding can happen between unrelated classes
C. False, overriding requires an inheritance (superclass-subclass) relationship
D. True, overriding only needs same class

Ans: C

Explanation: Overriding is fundamentally dependent on the existence of a superclass method being redefined by a subclass.

Q85 | Overriding COMPARISON

Overloading changes the _____, while overriding keeps the _____ the same.

A. parameter list; method signature
B. object; constructor
C. class; return type
D. method name; parameter list

Ans: A

Explanation: Overloading requires varying parameter lists with the same name, whereas overriding preserves the exact same signature.

Q86 | Overriding ASSERTION-REASON

Assertion (A): Overriding is resolved at compile time. Reason (R): The compiler always knows the exact runtime type of every object reference.

A. A is false, R is true
B. Both A and R are true, R explains A
C. A is true, R is false
D. Both A and R are false

Ans: D

Explanation: Overriding is resolved at runtime, and the compiler cannot generally know an object's exact runtime type in advance, so both statements are false.

Section 7 — Constructors

15 Questions

Q87 | Constructors DEFINITION

A constructor in Java is used to:

A. Declare a static variable
B. Override a method
C. Initialize a newly created object
D. Destroy an object

Ans: C

Explanation: Constructors run automatically when an object is created, setting up its initial state.

Q88 | Constructors RULE-BASED

A constructor's name must:

A. Be different from the class name
B. Exactly match the class name
C. Match the name of any method in the class
D. Start with a lowercase letter always

Ans: B

Explanation: Java requires a constructor's name to be identical to its class name.

Q89 | Constructors CONCEPTUAL

Constructors are invoked:

A. Only for static classes
B. Only once per class regardless of the number of objects
C. Automatically when an object is created using new
D. Only when explicitly called like a normal method

Ans: C

Explanation: The new keyword triggers the appropriate constructor automatically as part of object creation.

Q90 | Constructors RULE-BASED

Does a constructor have a return type?

A. Yes, constructors always return void
B. Yes, constructors return an int by default
C. No, constructors do not declare any return type, not even void
D. Yes, constructors return the class type explicitly

Ans: C

Explanation: Unlike methods, constructors have no return type declaration, which distinguishes them from methods.

Q91 | Constructors DEFINITION

A constructor that accepts arguments to initialize an object is called a:

A. Static constructor
B. Default constructor
C. Parameterized constructor
D. Overriding constructor

Ans: C

Explanation: A parameterized constructor takes input values used to set the initial state of the object.

Q92 | Constructors CONCEPTUAL

If a class defines no constructor at all, Java provides:

A. A no-argument default constructor automatically
B. A constructor that throws an exception
C. A static factory method
D. No way to create objects of that class

Ans: A

Explanation: The compiler supplies an implicit no-arg constructor when the class defines none explicitly.

Q93 | Constructors DEFINITION

Defining multiple constructors in the same class with different parameter lists is called:

A. Constructor finalization
B. Constructor overriding
C. Constructor overloading
D. Constructor inheritance

Ans: C

Explanation: Like methods, constructors can be overloaded by varying their parameter lists within the same class.

Q94 | Constructors IDENTIFICATION

```
class Point{ Point(){...} Point(int x, int y){...} } This class demonstrates:
```

A. Method overriding
B. Constructor overriding
C. Constructor overloading
D. Data hiding

Ans: C

Explanation: Two constructors with different parameter lists in the same class represent overloading.

Q95 | Constructors RULE-BASED

Can constructors be overridden in a subclass?

A. Yes, if declared final
B. Yes, exactly like normal methods
C. No, constructors are not inherited and therefore cannot be overridden
D. Yes, but only no-argument constructors

Ans: C

Explanation: Constructors are tied to their own class and are not inherited by subclasses.

Q96 | Constructors CODE/OUTPUT

```
class Box{ Box(){print('default');} Box(int s){print('sized');} } new Box(5); Output?
```

A. Compilation error
B. sized
C. default
D. sized then default

Ans: B

Explanation: Passing an argument 5 matches the parameterized constructor Box(int s).

Q97 | Constructors CONCEPTUAL

Which of these correctly relates constructors and object creation?

A. A constructor runs only if explicitly invoked by name later

B. A constructor runs before the object's memory is allocated

C. A constructor runs exactly once, right after memory is allocated for a new object

D. A constructor runs every time any method of the object is called

Ans: C

Explanation: Constructors execute automatically as the final step of object creation, running once per object.

Q98 | Constructors MISCONCEPTION

A student defines a method named the same as the class but adds a return type of void. Is this a constructor?

A. No, because constructor names cannot match the class name

B. Yes, it is still treated as a constructor

C. No, adding any return type (even void) makes it a regular method, not a constructor

D. Yes, but it will never be called automatically

Ans: C

Explanation: The absence of a return type is essential to constructor syntax; specifying void makes it an ordinary method.

Q99 | Constructors FILL IN THE BLANK

A constructor that takes no parameters is commonly called a _____ constructor.

A. abstract

B. static

C. final

D. no-argument (default)

Ans: D

Explanation: A constructor with an empty parameter list is referred to as a no-argument or default constructor.

Q100 | Constructors SCENARIO

A Rectangle class needs to support creating objects either with default dimensions or with specific width and height supplied by the caller. Which feature should be used?

A. Dynamic method dispatch

B. Constructor overloading

C. Method overriding

D. Data hiding

Ans: B

Explanation: Providing both a no-arg constructor and a parameterized constructor lets objects be created in multiple ways.

Q101 | Constructors ASSERTION-REASON

```
Assertion (A): If a class defines a parameterized constructor only, calling new ClassName() with no arguments will fail to compile. Reason (R): Java does not automatically provide a no-argument constructor once any constructor is explicitly defined.
```

A. A is false, R is true

B. Both A and R are true, and R correctly explains A

C. A is true, R is false

D. Both A and R are false

Ans: B

Explanation: Once a class defines any explicit constructor, the compiler no longer supplies a default no-arg constructor.

Section 8 — Finalization

9 Questions

Q102 | Finalization CONCEPT

The concept of finalization in Java relates to:

A. Actions performed immediately when an object is created
B. Declaring a class as final
C. Actions performed on an object before it is reclaimed by garbage collection
D. Compiling a class into bytecode

Ans: C

Explanation: Finalization refers to cleanup-related processing associated with an object nearing the end of its lifecycle.

Q103 | Finalization CONCEPT

The purpose of finalization is generally to:

A. Prevent objects from ever being deleted
B. Speed up object creation
C. Allow an object a chance to perform cleanup before it is removed from memory
D. Convert an object into its superclass

Ans: C

Explanation: Finalization gives an object an opportunity to release resources or perform final tasks before reclamation.

Q104 | Finalization CONCEPTUAL

Finalization is related to which part of an object's existence?

A. The middle of its lifecycle during normal method calls
B. It is unrelated to the object lifecycle
C. The very beginning, during construction
D. The end of its lifecycle, just before memory reclamation

Ans: D

Explanation: Finalization specifically concerns the object's lifecycle near its conclusion, prior to reclamation.

Q105 | Finalization CONCEPTUAL

When might finalization occur for an object?

A. Only when the program starts
B. When the object is no longer reachable and is being considered for garbage collection
C. Immediately after the constructor finishes
D. Every time a method is called on the object

Ans: B

Explanation: Finalization is associated with unreachable objects that are eligible for garbage collection.

Q106 | Finalization BASIC CHARACTERISTIC

Is the exact timing of finalization guaranteed to be immediate after an object becomes unreachable?

A. No, its timing is not guaranteed to be immediate
B. Yes, but only for static objects
C. No, it never happens at all
D. Yes, it always happens instantly

Ans: A

Explanation: Finalization timing is not precisely predictable; it depends on when the object is processed for reclamation.

Q107 | Finalization CONCEPTUAL

Finalization is best associated with which broader stage of object management?

A. Object creation
B. Method overloading
C. Constructor chaining
D. Object lifecycle conclusion / cleanup

Ans: D

Explanation: It represents the closing stage of an object's life, distinct from creation or normal usage.

Q108 | Finalization CONCEPT

Which statement about finalization is most accurate at a basic conceptual level?

A. It runs before an object is ever used
B. It is required for every object in Java
C. It provides a mechanism to perform final operations before an object is reclaimed
D. It replaces the need for constructors

Ans: C

Explanation: Finalization offers an optional opportunity for last operations, not a mandatory or initial step.

Q109 | Finalization FILL IN THE BLANK

Finalization is conceptually the counterpart to _____ in an object's lifecycle.

A. construction/initialization
B. polymorphism
C. overloading
D. encapsulation

Ans: A

Explanation: If construction begins an object's life, finalization is conceptually associated with its end.

Q110 | Finalization SCENARIO

A resource-holding object is no longer used anywhere in a program. Basic finalization concept suggests that:

A. The object is automatically overridden
B. The object is immediately converted to null
C. The object gains a new constructor
D. The object may get a chance to perform final cleanup before being reclaimed

Ans: D

Explanation: Once unreachable, an object may undergo finalization-related processing prior to being reclaimed.

Section 9 — Subclasses (Inheritance)

14 Questions

Q111 | Subclasses (Inheritance) DEFINITION

Inheritance in Java allows a class to:

A. Merge two unrelated classes permanently
B. Acquire the fields and methods of another class
C. Delete methods from another class
D. Prevent any class from being extended

Ans: B

Explanation: Inheritance lets a class (subclass) reuse and extend the members of another class (superclass).

Q112 | Subclasses (Inheritance) DEFINITION

The class that inherits from another class is called the:

A. Subclass
B. Constructor class
C. Superclass
D. Interface

Ans: A

Explanation: The inheriting class is termed the subclass (or derived/child class).

Q113 | Subclasses (Inheritance) DEFINITION

The class being inherited from is called the:

A. Final class
B. Superclass
C. Object class only
D. Subclass

Ans: B

Explanation: The class whose members are inherited is called the superclass (or base/parent class).

Q114 | Subclasses (Inheritance) CONCEPTUAL

The primary purpose of inheritance is to:

A. Promote code reuse by allowing subclasses to use superclass functionality
B. Prevent classes from sharing any behavior
C. Eliminate the need for constructors
D. Force every subclass to redefine all methods

Ans: A

Explanation: Inheritance enables a subclass to reuse existing fields and methods rather than rewriting them.

Q115 | Subclasses (Inheritance) RULE-BASED

Which members of a superclass are typically inherited by a subclass?

A. Only constructors
B. Only private fields
C. Only static methods
D. Non-private fields and methods

Ans: D

Explanation: A subclass generally inherits accessible (non-private) fields and methods; constructors are not inherited.

Q116 | Subclasses (Inheritance) CONCEPTUAL

The relationship established by inheritance between a subclass and superclass is often described as:

A. A completely independent relationship
B. A parent-child (is-a) relationship
C. A uses-a relationship exclusively
D. A has-a relationship exclusively

Ans: B

Explanation: Inheritance models an 'is-a' relationship, where a subclass object is a specialized kind of the superclass.

Q117 | Subclasses (Inheritance) CONCEPTUAL

Which of these is an advantage of using inheritance?

A. It always increases the number of unrelated classes needed
B. It reduces code duplication by reusing existing class functionality
C. It prevents subclasses from adding new functionality
D. It removes the need for any class to have methods

Ans: B

Explanation: Inheritance's core benefit is reusing established behavior, reducing redundant code across related classes.

Q118 | Subclasses (Inheritance) IDENTIFICATION

```
class Vehicle{...} class Car extends Vehicle{...} Car is the:
```

A. Constructor
B. Superclass
C. Interface
D. Subclass

Ans: D

Explanation: Car extends Vehicle, making Car the subclass that inherits from the superclass Vehicle.

Q119 | Subclasses (Inheritance) RULE-BASED

Which keyword is used in Java to establish inheritance between classes?

A. implements
B. super
C. extends
D. inherits

Ans: C

Explanation: The extends keyword is used when a subclass inherits from a superclass in Java.

Q120 | Subclasses (Inheritance) BASIC CHARACTERISTIC

```
Can a subclass add new fields and methods not present in its superclass?
```

A. No, a subclass can only use existing superclass members
B. No, a subclass must have identical members to its superclass
C. Yes, but only new fields, never new methods
D. Yes, a subclass can extend the superclass by adding its own new members

Ans: D

Explanation: In addition to inheriting members, subclasses can introduce entirely new fields and methods.

Q121 | Subclasses (Inheritance) SCENARIO

A Manager class extends an Employee class, reusing salary calculation logic while adding a teamSize field. This illustrates:

A. Overloading
B. Data hiding
C. Finalization
D. Inheritance

Ans: D

Explanation: Manager reuses Employee's functionality while extending it with new members, which is inheritance.

Q122 | Subclasses (Inheritance) FILL IN THE BLANK

In an inheritance relationship, the subclass is also referred to as the _____ class.

A. static
B. final
C. abstract
D. derived (or child)

Ans: D

Explanation: Subclass, derived class, and child class are commonly interchangeable terms for the inheriting class.

Q123 | Subclasses (Inheritance) CONCEPTUAL

Which statement about parent-child class relationships is correct?

A. A child class can have only one method
B. A child class inherits characteristics from its parent class but may also specialize them
C. A parent class inherits from its child class
D. Parent and child classes must always be unrelated in behavior

Ans: B

Explanation: The child (subclass) builds upon its parent's (superclass's) behavior, often specializing or extending it.

Q124 | Subclasses (Inheritance) COMPARISON

Superclass vs Subclass - which best characterizes their typical relationship?

A. The superclass is always smaller in code size
B. The subclass always has fewer methods than the superclass
C. They have no hierarchical relationship
D. The superclass is more general; the subclass is more specific

Ans: D

Explanation: Superclasses typically represent general concepts, while subclasses specialize or refine that general behavior.

Section 10 — Superclass and Subclass Object Relationship

11 Questions

Q125 | Superclass and Subclass Object Relationship CONCEPT

A subclass object can generally be treated as an object of its:

A. Only its own class, never another
B. Interface it does not implement
C. Superclass
D. Any unrelated class

Ans: C

Explanation: Because of the is-a relationship, a subclass object is also considered an object of its superclass type.

Q126 | Superclass and Subclass Object Relationship CONCEPTUAL

Is every superclass object also a subclass object?

A. No, a superclass object is not necessarily a subclass object
B. Yes, always
C. Yes, but only for abstract superclasses
D. No such relationship exists

Ans: A

Explanation: The is-a relationship is one-directional: subclass objects are superclass objects, but not the reverse.

Q127 | Superclass and Subclass Object Relationship RULE-BASED

A reference variable of the superclass type can hold:

A. An object of the superclass or any of its subclasses
B. Only an object of the exact same class
C. Nothing except null under all circumstances
D. Only objects of unrelated classes

Ans: A

Explanation: Superclass references are flexible enough to point to subclass instances, reflecting the inheritance hierarchy.

Q128 | Superclass and Subclass Object Relationship RULE-BASED

Can a superclass reference variable hold a subclass object directly through assignment?

A. Yes, but only inside static methods
B. No, this always causes a compile-time error
C. Yes, this assignment is allowed and considered valid
D. No, this always causes a runtime error

Ans: C

Explanation: Assigning a subclass object to a superclass reference is a fundamental, valid part of Java's type compatibility rules.

Q129 | Superclass and Subclass Object Relationship CONCEPTUAL

Type compatibility in inheritance means that:

A. Only identical classes are type-compatible
B. A superclass type is compatible with any subclass type without restriction
C. A subclass type is compatible with (can be treated as) its superclass type
D. No two related classes are ever type-compatible

Ans: C

Explanation: Subclass types are compatible with their superclass because they inherit and extend its structure.

Q130 | Superclass and Subclass Object Relationship CODE/OUTPUT

```
class A{} class B extends A{} A ref = new B(); Is this valid?
```

A. Only valid if A is abstract
B. No, this is a compile-time error
C. No, this is a runtime error
D. Yes, valid because B is a subclass of A

Ans: D

Explanation: Assigning a B object to an A-typed reference is valid due to the inheritance relationship between A and B.

Q131 | Superclass and Subclass Object Relationship COMPARISON

Which of these correctly compares superclass and subclass objects?

A. A subclass object generally contains all superclass members plus its own additional members
B. A subclass object contains fewer members than its superclass object
C. A subclass object and superclass object are always identical in structure
D. A subclass object cannot have any members of its own

Ans: A

Explanation: Because of inheritance, subclass objects extend the superclass structure with additional state and behavior.

Q132 | Superclass and Subclass Object Relationship CONCEPTUAL

A subclass object 'is-a' superclass object; this relationship supports:

A. Assigning subclass objects to superclass references
B. Removing the superclass entirely from memory
C. Preventing any use of inherited methods
D. Assigning superclass objects freely to subclass references without restriction

Ans: A

Explanation: The is-a relationship is precisely what allows subclass instances to be referenced using superclass-typed variables.

Q133 | Superclass and Subclass Object Relationship BASIC RELATIONSHIP

Which best describes the object relationship in inheritance?

A. A subclass object and superclass object are entirely unrelated at runtime
B. A subclass object can be viewed as a more specific version of a superclass object
C. Subclass and superclass objects never share any relationship
D. A superclass object is a more specific version of a subclass object

Ans: B

Explanation: The subclass specializes the superclass, so its objects represent more specific instances of the general superclass type.

Q134 | Superclass and Subclass Object Relationship OUTPUT/BEHAVIOR

Animal a = new Dog(); What can be said about 'a' at this point?

A. This line always fails to compile
B. a is a reference of type Animal referring to an actual Dog object
C. a cannot refer to a Dog object at all
D. a is a Dog reference referring to an Animal object

Ans: B

Explanation: The reference type is Animal, but the actual object it points to is a Dog, which is valid due to inheritance.

Q135 | Superclass and Subclass Object Relationship ASSERTION-REASON

Assertion (A): A superclass reference can always call any method defined newly in the subclass. Reason (R): The relationship between superclass and subclass objects makes all subclass members accessible via superclass references.

A. Both A and R are true, R explains A

B. Both A and R are false

C. A is true, R is false

D. A is false, R is true

Ans: B

Explanation: A superclass reference can only directly call methods declared in the superclass (or overridden ones), not new subclass-only methods.

Section 11 — Implicit Subclass-to-Superclass Conversion

11 Questions

Q136 | Implicit Subclass-to-Superclass Conversion DEFINITION

Assigning a subclass object to a superclass reference without any explicit cast is known as:

A. Implicit upcasting
B. Dynamic casting
C. Explicit downcasting
D. Object finalization

Ans: A

Explanation: This automatic, cast-free conversion from subclass to superclass type is called implicit upcasting.

Q137 | Implicit Subclass-to-Superclass Conversion RULE-BASED

Is a cast operator required to assign a subclass object to a superclass reference?

A. Yes, an explicit cast is always mandatory
B. No, this conversion happens automatically without an explicit cast
C. Yes, but only for abstract superclasses
D. No such assignment is permitted at all

Ans: B

Explanation: Because it is a widening (upcasting) conversion, Java performs it implicitly without requiring a cast.

Q138 | Implicit Subclass-to-Superclass Conversion IDENTIFICATION

Which of the following correctly demonstrates implicit conversion (Animal is the superclass of Dog)?

A. Dog d = new Animal();
B. Animal a = new Dog();
C. Object o = (Dog) new Animal();
D. Dog d = new Object();

Ans: B

Explanation: Assigning a Dog object directly to an Animal reference is a valid, implicit upcasting conversion.

Q139 | Implicit Subclass-to-Superclass Conversion CONCEPTUAL

Why is subclass-to-superclass conversion considered 'safe' and automatic?

A. Because Java does not check type compatibility at all
B. Because superclass objects are always smaller in memory
C. Because a subclass object always contains everything a superclass object has, and more
D. Because subclasses never add new members

Ans: C

Explanation: Since the subclass is guaranteed to include the superclass's structure, treating it as a superclass instance is inherently safe.

Q140 | Implicit Subclass-to-Superclass Conversion VALID/INVALID

Is 'Object obj = new String("hi");' a valid implicit conversion?

A. No, this requires an explicit cast
B. Yes, but only inside static methods
C. No, Strings cannot be converted to Object
D. Yes, since String is (indirectly) a subclass of Object

Ans: D

Explanation: Every class in Java is ultimately a subclass of Object, so assigning any object to an Object reference is valid.

Q141 | Implicit Subclass-to-Superclass Conversion MISCONCEPTION

A student believes implicit conversion works both ways (superclass object to subclass reference) without a cast. Is this correct?

A. Yes, both directions work implicitly

B. Yes, but only for final classes

C. No, neither direction is ever allowed

D. No, converting a superclass reference to a subclass reference requires an explicit cast (downcasting)

Ans: D

Explanation: Only subclass-to-superclass (upcasting) is implicit; the reverse (downcasting) needs an explicit cast.

Q142 | Implicit Subclass-to-Superclass Conversion RULE-BASED

After implicit conversion, a superclass reference variable pointing to a subclass object can directly access:

A. Only the members defined in the superclass (plus overridden methods behaving polymorphically)

B. No members at all

C. All members defined anywhere in the subclass without restriction

D. Only static members of the subclass

Ans: A

Explanation: The reference type restricts which members are directly accessible, even though the actual object is of the subclass type.

Q143 | Implicit Subclass-to-Superclass Conversion OUTPUT/BEHAVIOR

```
class A{void show(){print('A');}} class B extends A{void show(){print('B');} void extra(){print('extra');}}
A ref = new B(); ref.show(); Output?
```

A. Compilation error

B. extra

C. A

D. B

Ans: D

Explanation: Even via implicit conversion to type A, the overridden show() executes according to the actual object's class, B.

Q144 | Implicit Subclass-to-Superclass Conversion RULE-BASED

Attempting ref.extra() where ref is declared as type A but refers to a B object (extra() only exists in B) results in:

A. It runs normally without issue

B. A runtime-only error with no compile issue

C. A compile-time error, since A's reference type doesn't declare extra()

D. extra() executes as if it were inherited

Ans: C

Explanation: The compiler checks accessibility based on the reference's declared type, so this fails to compile.

Q145 | Implicit Subclass-to-Superclass Conversion FILL IN THE BLANK

Assigning a subclass object to a superclass reference without a cast is known as _____.

A. narrowing

B. upcasting

C. downcasting

D. finalizing

Ans: B

Explanation: This automatic, safe conversion is specifically termed upcasting.

Q146 | Implicit Subclass-to-Superclass Conversion ASSERTION-REASON

Assertion (A): Implicit subclass-to-superclass conversion can lose access to subclass-specific members through that reference. Reason (R): The compiler restricts accessible members based on the reference's declared type.

A. A is true, R is false

B. Both A and R are true, and R correctly explains A

C. A is false, R is true

D. Both are false

Ans: B

Explanation: Because member accessibility is determined by the declared reference type, subclass-specific members become inaccessible directly through it.

Section 12 — Dynamic Method Dispatch

14 Questions

Q147 | Dynamic Method Dispatch DEFINITION

Dynamic method dispatch refers to:

A. Converting primitives to objects
B. Resolving overloaded methods at compile time
C. Assigning values to instance variables
D. The mechanism by which an overridden method call is resolved at runtime based on the actual object's type

Ans: D

Explanation: Dynamic method dispatch determines, at runtime, which overridden method implementation actually executes.

Q148 | Dynamic Method Dispatch CONCEPTUAL

Dynamic method dispatch is closely tied to which concept?

A. Method overriding
B. Method overloading
C. Data hiding
D. Constructor chaining

Ans: A

Explanation: This mechanism specifically applies to overridden methods, selecting the correct version based on the object's actual class.

Q149 | Dynamic Method Dispatch RULE-BASED

When a superclass reference refers to a subclass object and an overridden method is called, which version executes?

A. The subclass's overridden version
B. Neither version executes
C. Both versions execute in sequence
D. The superclass's version always

Ans: A

Explanation: Dynamic dispatch selects the method implementation belonging to the actual runtime object, which is the subclass.

Q150 | Dynamic Method Dispatch CODE/OUTPUT

```
class A{void greet(){print('Hello from A');}} class B extends A{void greet(){print('Hello from B');}} A obj = new B(); obj.greet(); Output?
```

A. Hello from A
B. Hello from A then Hello from B
C. Hello from B
D. Compilation error

Ans: C

Explanation: Despite the reference type being A, the actual object is B, so dynamic dispatch invokes B's overridden greet().

Q151 | Dynamic Method Dispatch CONCEPTUAL

Method selection for dynamic dispatch occurs:

A. At compile time
B. At runtime
C. Never, it is a static decision
D. During class loading only, never after

Ans: B

Explanation: The defining feature of dynamic dispatch is that method resolution happens while the program is running.

Q152 | Dynamic Method Dispatch COMPARISON

Static (compile-time) method selection differs from dynamic method dispatch in that:

A. Dynamic dispatch only applies to constructors

B. Static selection is decided by the compiler using reference type; dynamic dispatch is decided at runtime using the actual object type

C. Static selection always overrides dynamic dispatch results

D. They are identical processes with different names

Ans: B

Explanation: Overload resolution (static) depends on declared types at compile time, whereas dynamic dispatch depends on the real object type at runtime.

Q153 | Dynamic Method Dispatch CONCEPTUAL

Dynamic method dispatch allows a program to:

A. Always execute the superclass version of a method

B. Skip calling constructors during object creation

C. Exhibit different behavior for the same method call depending on the actual object type

D. Prevent any subclass from overriding methods

Ans: C

Explanation: This runtime flexibility is exactly what enables polymorphic behavior through a common reference type.

Q154 | Dynamic Method Dispatch CODE/OUTPUT

```
class Shape{void draw(){print('Shape');}} class Circle extends Shape{void draw(){print('Circle');}} class Square extends Shape{void draw(){print('Square');}} Shape s; s=new Circle(); s.draw(); s=new Square(); s.draw(); Output (two lines)?
```

A. Circle then Square

B. Circle then Circle

C. Shape then Shape

D. Square then Square

Ans: A

Explanation: Each time, dynamic dispatch selects the draw() method matching the object currently assigned to s.

Q155 | Dynamic Method Dispatch RULE-BASED

Which is a requirement for dynamic method dispatch to apply to a method call?

A. The method must be static

B. The method must be overridden in the subclass with the same signature

C. The method must be overloaded, not overridden

D. The method must be declared final

Ans: B

Explanation: Dynamic dispatch only applies to overriding relationships; static and final methods are resolved differently.

Q156 | Dynamic Method Dispatch MISCONCEPTION

A student thinks the reference type alone determines which overridden method executes. Is this correct?

A. No, the actual object type (not the reference type) determines which overridden method executes

B. Yes, the reference type always determines it

C. No, the method executed is chosen randomly

D. Yes, but only for static methods

Ans: A

Explanation: Dynamic dispatch bases its decision on the object's real, runtime type - not merely its reference's declared type.

Q157 | Dynamic Method Dispatch EXECUTION BEHAVIOR

In dynamic method dispatch, which factor is primarily responsible for selecting the correct overridden method at runtime?

A. The garbage collector
B. The constructor of the superclass
C. The runtime object's actual type information
D. The compiler's static type checker only

Ans: C

Explanation: The JVM uses the actual object's runtime type to look up and invoke the correct overridden method version.

Q158 | Dynamic Method Dispatch SCENARIO

A drawing application stores various shape objects and calls draw() on each through a common Shape-typed reference without knowing their specific subclass. This relies on:

A. Constructor overloading
B. Pass by value
C. Dynamic method dispatch
D. Data hiding

Ans: C

Explanation: Calling draw() polymorphically on different subclass objects through a common superclass type is the classic use case for dynamic dispatch.

Q159 | Dynamic Method Dispatch FILL IN THE BLANK

Dynamic method dispatch allows a superclass reference to invoke a method whose specific implementation is determined by the object's _____ type.

A. runtime (actual)
B. static
C. compile-time
D. declared

Ans: A

Explanation: The runtime (actual) type of the referenced object governs which overridden method version executes.

Q160 | Dynamic Method Dispatch ASSERTION-REASON

Assertion (A): Overloaded methods are also resolved through dynamic method dispatch. Reason (R): Overload resolution depends on the compile-time reference type, not the runtime object type.

A. A is true, R is false
B. A is false, R is true
C. Both are false
D. Both A and R are true, R explains A

Ans: B

Explanation: Overloading is resolved at compile time based on reference types, so it does not use dynamic dispatch.

Section 13 — Comparison Questions

12 Questions

Q161 | Comparison Questions COMPARISON

Object vs Class - which statement is correct?

A. Objects and classes are the same thing viewed differently

B. A class can only produce one object

C. An object is a blueprint; a class is a runtime instance

D. A class is a blueprint; an object is a runtime instance created from that blueprint

Ans: D

Explanation: The class defines structure and behavior; objects are the actual instances created based on that definition.

Q162 | Comparison Questions COMPARISON

Pass by Value vs 'reference-like' behavior - which is accurate for Java?

A. Java has no consistent parameter-passing mechanism

B. Java is strictly pass by value, even though object reference values can make it appear as if references are passed

C. Java uses pass by reference for objects and pass by value for primitives

D. Java uses pass by reference for all types

Ans: B

Explanation: Java uniformly copies values (including reference values) when passing arguments, which is technically always pass by value.

Q163 | Comparison Questions COMPARISON

Data Hiding vs Encapsulation - how are they best distinguished?

A. They are entirely unrelated concepts with no overlap

B. Data hiding is the restriction of access to data; encapsulation is the broader bundling of data and methods with controlled access

C. Encapsulation only applies to constructors

D. Data hiding is broader than encapsulation

Ans: B

Explanation: Encapsulation is the general design principle; data hiding is the protective restriction of direct data access it enables.

Q164 | Comparison Questions COMPARISON

Overloading vs Overriding - key structural difference?

A. Overloading occurs within the same class with differing parameter lists; overriding occurs across superclass-subclass with identical signatures

B. Both occur only within the same class

C. Overloading occurs across classes; overriding occurs within one class

D. Both require different parameter lists

Ans: A

Explanation: This is the fundamental structural distinction between the two forms of polymorphism-related features.

Q165 | Comparison Questions COMPARISON

Constructor vs Method - which is a key difference?

A. A constructor always has a return type; a method never does

B. A constructor has no return type and shares the class's name; a method has a return type and can have any valid name

C. Constructors and methods are identical in every respect

D. Methods must share the class name; constructors need not

Ans: B

Explanation: The absence of a return type and the mandatory class-name match are defining features that separate constructors from methods.

Q166 | Comparison Questions COMPARISON

Superclass vs Subclass - which best captures their relationship?

A. A subclass is always more general than its superclass

B. A superclass depends on its subclass for its own definition

C. They must always have identical members

D. A superclass is more general; a subclass extends and specializes it

Ans: D

Explanation: Inheritance flows from a general superclass to a more specific subclass, which extends the general behavior.

Q167 | Comparison Questions COMPARISON

Superclass reference vs Subclass reference - accessing subclass-specific members:

A. A superclass reference has broader access than a subclass reference

B. Neither reference type can access subclass-only members

C. Both reference types can access all subclass members equally

D. A superclass reference cannot directly access members declared only in the subclass, while a subclass reference can

Ans: D

Explanation: The declared reference type restricts which members are directly accessible; superclass references are limited to superclass-declared members.

Q168 | Comparison Questions COMPARISON

Static (compile-time) method selection vs Dynamic Method Dispatch - how do they differ?

A. Both occur exclusively at compile time

B. Static selection uses reference type at compile time (as in overloading); dynamic dispatch uses actual object type at runtime (as in overriding)

C. Both occur exclusively at runtime

D. Static selection applies to overriding, dynamic dispatch applies to overloading

Ans: B

Explanation: This is the essential contrast: compile-time resolution relies on declared types, while dynamic dispatch relies on the object's real runtime type.

Q169 | Comparison Questions COMPARISON

Which comparison correctly relates encapsulation and overloading?

A. Overloading always breaks encapsulation

B. Encapsulation and overloading are synonyms

C. Overloading is a technique used exclusively to achieve encapsulation

D. They address different concerns entirely: encapsulation is about controlling access to data, while overloading is about providing multiple method versions with the same name

Ans: D

Explanation: These two concepts serve entirely different purposes in object-oriented design and are not directly related mechanisms.

Q170 | Comparison Questions COMPARISON

Constructors vs Overriding - why can't constructors be overridden?

A. Because constructors are not inherited by subclasses at all

B. Because constructors have no name

C. Because subclasses are not allowed to define any constructors

D. Because constructors are always declared final

Ans: A

Explanation: Since inheritance does not extend to constructors, the concept of overriding cannot apply to them.

Q171 | Comparison Questions COMPARISON

Object vs Class - in terms of memory, which statement is correct?

A. A class and its objects always occupy identical memory

B. Objects consume no memory at all

C. Only classes consume memory, never their objects

D. A class definition itself does not consume memory the way an instantiated object does

Ans: D

Explanation: The class serves as a compiled template, while each object created from it occupies its own memory for its state.

Q172 | Comparison Questions COMPARISON

Which correctly compares subclass-to-superclass conversion direction with the reverse?

A. Both directions require an explicit cast

B. Subclass-to-superclass conversion is implicit; superclass-to-subclass conversion requires an explicit cast

C. Subclass-to-superclass requires a cast; the reverse does not

D. Both directions are implicit

Ans: B

Explanation: Upcasting (subclass to superclass) is automatic, while downcasting (superclass to subclass) must be explicitly cast.

Section 14 — Code and Output-Based Questions

14 Questions

Q173 | Code and Output-Based Questions CODE/OUTPUT

```
class Emp{ String name; Emp(String n){this.name=n;} } Emp e = new Emp('Rita'); print(e.name); Output?
```

A. Emp
B. Compilation error
C. null
D. Rita

Ans: D

Explanation: The constructor uses this.name to assign the parameter to the instance variable, so e.name correctly holds 'Rita'.

Q174 | Code and Output-Based Questions CODE/OUTPUT

```
class Counter{ int c=0; void inc(){c++;} } Counter x=new Counter(); Counter y=x; y.inc(); print(x.c);
Output?
```

A. 1
B. Compilation error
C. 2
D. 0

Ans: A

Explanation: x and y reference the same object, so incrementing through y also affects the object x refers to.

Q175 | Code and Output-Based Questions WHICH METHOD INVOKED

```
class P{void m(){print('P.m');}} class Q extends P{void m(){print('Q.m');}} P obj=new Q(); obj.m(); Which
method executes?
```

A. Q's m() method
B. P's m() method
C. Both execute
D. Neither executes, compile error

Ans: A

Explanation: Dynamic method dispatch selects Q's overridden m() based on the actual object type.

Q176 | Code and Output-Based Questions WHICH CONSTRUCTOR EXECUTES

```
class Item{ Item(){print('no-arg');} Item(int q){print('qty:'+q);} } new Item(3); Output?
```

A. no-arg
B. qty:3
C. Compilation error
D. qty:3 then no-arg

Ans: B

Explanation: The argument 3 matches the parameterized constructor Item(int q), so that constructor runs.

Q177 | Code and Output-Based Questions WHICH OBJECT CREATED

```
class A{} class B extends A{} A ref = new B(); What type of object does ref actually refer to?
```

A. An object of class A
B. No object, only a reference
C. An object of class B
D. An object of class Object

Ans: C

Explanation: Although declared as type A, ref is assigned a newly created B object via new B().

Q178 | Code and Output-Based Questions WHAT DOES 'THIS' REFER TO

```
class Car{ String model; Car(String model){ this.model = model; } String getModel(){ return this.model; } }
Inside getModel(), this refers to:
```

A. A static variable
B. The Car class itself
C. The Car object on which getModel() was called
D. The parameter model

Ans: C

Explanation: this always denotes the specific object instance currently executing the method.

Q179 | Code and Output-Based Questions OVERLOADED OR OVERRIDDEN

```
class Calc{ int add(int a,int b){return a+b;} double add(double a,double b){return a+b;} } The two add()
methods are:
```

A. Overridden
B. Overloaded
C. Neither
D. Both overloaded and overridden

Ans: B

Explanation: Both methods exist in the same class with different parameter types, which defines overloading.

Q180 | Code and Output-Based Questions SUBCLASS TO SUPERCLASS REFERENCE

```
class Bird{void fly(){print('flies');}} class Sparrow extends Bird{} Bird b = new Sparrow(); b.fly(); What
happens?
```

A. Nothing is printed
B. Runtime error
C. It prints 'flies' since Sparrow inherits fly() from Bird
D. Compilation error since Sparrow has no fly()

Ans: C

Explanation: Sparrow does not override fly(), so it inherits and uses Bird's implementation directly.

Q181 | Code and Output-Based Questions METHOD SELECTED IN DYNAMIC DISPATCH

```
class Emp{double salary(){return 1000;}} class Manager extends Emp{double salary(){return 2000;}} Emp e=new
Manager(); print(e.salary()); Output?
```

A. 1000.0
B. 3000.0
C. Compilation error
D. 2000.0

Ans: D

Explanation: Dynamic method dispatch invokes Manager's overridden salary() since the actual object is a Manager.

Q182 | Code and Output-Based Questions VALID CONVERSION CHECK

```
class X{} class Y extends X{} X obj1 = new Y(); Y obj2 = new X(); Which line is invalid?
```

A. Both lines are invalid
B. X obj1 = new Y();
C. Neither line is invalid
D. Y obj2 = new X();

Ans: D

Explanation: Assigning a superclass object (X) to a subclass reference (Y) requires an explicit cast; the first line is valid implicit upcasting.

Q183 | Code and Output-Based Questions VALUE PASSED TO METHOD

```
void update(int val){ val = val * 2; } int num = 10; update(num); print(num); Output?
```

A. 20
B. Compilation error
C. 0
D. 10

Ans: D

Explanation: Since int is a primitive, only a copy of num's value is passed, so changes inside update() do not affect num.

Q184 | Code and Output-Based Questions CODE/OUTPUT

```
class Point{int x,y; Point(int x,int y){this.x=x; this.y=y;} void show(){print(this.x+','+this.y);}} new Point(3,4).show(); Output?
```

A. 3,4
B. Compilation error
C. 0,0
D. 4,3

Ans: A

Explanation: The constructor assigns parameters to instance variables via this, and show() prints them in the same order.

Q185 | Code and Output-Based Questions CODE/OUTPUT

```
class A{int x=1;} class B extends A{void show(){print(x);}} new B().show(); Output?
```

A. null
B. Compilation error
C. 0
D. 1

Ans: D

Explanation: B inherits the field x from A with value 1, and show() correctly prints the inherited value.

Q186 | Code and Output-Based Questions PROGRAM-TRACING

```
class Base{Base(){print('Base');} void greet(){print('Base greet');}} class Derived extends Base{Derived(){print('Derived');} void greet(){print('Derived greet');}} Base b=new Derived(); b.greet(); Output (in order)?
```

A. Derived, then Base, then Base greet
B. Base greet, then Derived greet
C. Base, then Derived, then Derived greet
D. Base, then Derived, then Base greet

Ans: C

Explanation: Object creation runs the superclass constructor first, then the subclass constructor; afterward, dynamic dispatch invokes Derived's overridden greet().

Section 15 — Difficult and Tricky Questions

12 Questions

Q187 | Difficult and Tricky Questions TRICKY

```
class A{ void m(){print('A');} } class B extends A{ void m(){print('B'); super.m();} } new B().m(); Output?
```

A. B
B. A, then B
C. A
D. B, then A

Ans: D

Explanation: B's m() first prints 'B', then explicitly calls the superclass version using super.m(), printing 'A'.

Q188 | Difficult and Tricky Questions TRICKY

```
class Data{int val;} void modify(Data d){ d.val = 50; d = new Data(); d.val = 100; } Data obj = new Data();
obj.val=1; modify(obj); print(obj.val); Output?
```

A. 1
B. Compilation error
C. 50
D. 100

Ans: C

Explanation: The first modification (d.val=50) affects the shared object; reassigning d afterward does not affect obj, so obj.val remains 50.

Q189 | Difficult and Tricky Questions TRICKY

```
class A{ int x; A(){ this(10); } A(int x){ this.x=x; } } What is the value of new A().x?
```

A. Undefined
B. Compilation error
C. 10
D. 0

Ans: C

Explanation: The no-arg constructor uses this(10) to invoke the parameterized constructor, setting x to 10.

Q190 | Difficult and Tricky Questions TRICKY

```
void test(int a, long b){} void test(long a, int b){} test(5,5); Result?
```

A. Calls test(long,int)
B. Compilation error due to ambiguous method call
C. Calls test(int,long)
D. Both methods execute

Ans: B

Explanation: Since 5 can widen to either position equally well, the compiler cannot determine a single best match, causing ambiguity.

Q191 | Difficult and Tricky Questions TRICKY

```
class A{ protected void show(){} } class B extends A{ private void show(){} } Is this valid overriding?
```

A. Yes, but only for private methods
B. No, an overriding method cannot reduce the visibility of the overridden method
C. Yes, subclasses can freely reduce visibility
D. No, because show() must be static

Ans: B

Explanation: Java requires the overriding method's access level to be the same or more accessible, never more restrictive.

Q192 | Difficult and Tricky Questions TRICKY

```
class A{ int x=1; void show(){print(x);} } class B extends A{ int x=2; void show(){print(x);} } A obj=new B(); obj.show(); print(obj.x); Output (two lines)?
```

A. 2, then 1
B. 1, then 2
C. 1, then 1
D. 2, then 2

Ans: A

Explanation: show() is overridden, so dynamic dispatch prints B's x (2); field access is resolved by reference type at compile time, so obj.x prints A's x (1).

Q193 | Difficult and Tricky Questions TRICKY

```
class A{ A(){print('A-init');} } class B extends A{ B(){print('B-init');} } new B(); What is the output order?
```

A. A-init, then B-init
B. B-init, then A-init
C. B-init only
D. A-init only

Ans: A

Explanation: A subclass constructor implicitly calls the superclass constructor first before executing its own body.

Q194 | Difficult and Tricky Questions TRICKY

```
class A{ void common(){} } class B extends A{ void common(){} void special(){} } A ref=new B(); ref.special(); Result?
```

A. Compilation error: special() is not accessible through an A-typed reference
B. Runs normally, printing nothing
C. Runs special() from B via dynamic dispatch
D. Runtime error only

Ans: A

Explanation: The compiler restricts accessible methods based on the declared reference type (A), which does not declare special().

Q195 | Difficult and Tricky Questions TRICKY

Assertion (A): An object is guaranteed to undergo finalization the moment it becomes unreachable.
Reason (R): Finalization timing is controlled entirely by explicit programmer calls.

A. A is false, R is true
B. Both A and R are true, R explains A
C. Both A and R are false
D. A is true, R is false

Ans: C

Explanation: Finalization is not guaranteed to occur immediately upon unreachability, and its timing is not directly controlled by explicit programmer calls.

Q196 | Difficult and Tricky Questions TRICKY

```
class A{ void calc(int x){print('A-int');} } class B extends A{ void calc(int x){print('B-int');} void calc(double x){print('B-double');} } A obj=new B(); obj.calc(5); Output?
```

A. Compilation error
B. B-double
C. B-int
D. A-int

Ans: C

Explanation: calc(int) in B overrides A's version; since 5 is an int, dynamic dispatch resolves to B's overriding calc(int).

Q197 | Difficult and Tricky Questions TRICKY

```
class A{ void run(){ print('A.run'); helper(); } void helper(){print('A.helper');} } class B extends A{ void helper(){print('B.helper');} } A obj=new B(); obj.run(); Output (two lines)?
```

A. A.run, then B.helper
B. A.run only
C. A.run, then A.helper
D. B.run, then B.helper

Ans: A

Explanation: run() executes as defined in A, but its internal call to helper() is dynamically dispatched to B's overridden version.

Q198 | Difficult and Tricky Questions TRICKY

```
class Shape{ String name(){return 'Shape';} } class Circle extends Shape{ String name(){return 'Circle';} } Shape s1=new Shape(); Shape s2=new Circle(); print(s1.name()+','+s2.name()); Output?
```

A. Shape,Shape
B. Circle,Circle
C. Shape,Circle
D. Circle,Shape

Ans: C

Explanation: s1 is an actual Shape object printing 'Shape'; s2 is a Circle object accessed via a Shape reference, and dynamic dispatch resolves to Circle's name().

Section 16 — Mixed OAVS TGT Practice Set

12 Questions

Q199 | Mixed OAVS TGT Practice Set APPLICATION-BASED

Which OOP feature allows a Manager object to be treated as an Employee object without any cast, assuming Manager extends Employee?

A. Finalization
B. Downcasting
C. Overloading
D. Implicit subclass-to-superclass conversion (upcasting)

Ans: D

Explanation: Since Manager is a subclass of Employee, this automatic conversion is implicit upcasting.

Q200 | Mixed OAVS TGT Practice Set IDENTIFICATION

A class Book has two constructors: one with no arguments and one accepting title and author. This design choice illustrates:

A. Method overriding
B. Dynamic method dispatch
C. Data hiding
D. Constructor overloading

Ans: D

Explanation: Multiple constructors with differing parameter lists in the same class represent constructor overloading.

Q201 | Mixed OAVS TGT Practice Set CONCEPT

Which keyword lets a method or constructor refer to the object it belongs to?

A. this
B. super
C. static
D. new

Ans: A

Explanation: this is Java's reference to the currently executing object.

Q202 | Mixed OAVS TGT Practice Set IDENTIFICATION

```
A subclass method with the same name and parameters as a superclass method, providing new behavior, demonstrates:
```

A. Constructor chaining
B. Method overloading
C. Object finalization
D. Method overriding

Ans: D

Explanation: Redefining an inherited method's behavior with an identical signature is overriding.

Q203 | Mixed OAVS TGT Practice Set CONCEPT

Which principle protects an object's internal fields from uncontrolled outside access?

A. Data hiding (through encapsulation)
B. Constructor overloading
C. Overloading
D. Dynamic method dispatch

Ans: A

Explanation: Data hiding restricts direct access, typically achieved via encapsulation using private fields and public methods.

Q204 | Mixed OAVS TGT Practice Set CONCEPT

When a superclass reference calls an overridden method on a subclass object, which mechanism determines the method that runs?

A. Static binding
B. Dynamic method dispatch
C. Pass by value
D. Constructor chaining

Ans: B

Explanation: The actual object's type governs which overridden method executes at runtime.

Q205 | Mixed OAVS TGT Practice Set IDENTIFICATION

Two methods in the same class named calculate(), one taking an int and another taking a String, represent:

A. A compilation error
B. Valid method overloading
C. Invalid overloading due to different types
D. Method overriding

Ans: B

Explanation: Differing parameter types with the same method name in one class is a valid overload.

Q206 | Mixed OAVS TGT Practice Set APPLICATION-BASED

If a Student object's roll number is changed inside a method that received the object as a parameter, why does this change persist after the method returns?

A. Because Java passes objects by reference, not value
B. Because the parameter is a copy of the reference, which still points to the same original object
C. Because the method automatically returns the object
D. Because roll number is a static field

Ans: B

Explanation: The copied reference and the original both point to the same object, so field changes are visible outside; passing itself remains by value.

Q207 | Mixed OAVS TGT Practice Set CONCEPT

A subclass inherits members from its superclass primarily to achieve:

A. Making all methods private
B. Preventing object creation
C. Removing constructors from the hierarchy
D. Reusability of existing code and behavior

Ans: D

Explanation: Inheritance's main goal is enabling reuse of already-defined class functionality.

Q208 | Mixed OAVS TGT Practice Set CONCEPT

Which of these accurately reflects finalization's basic role?

A. It offers an object a chance for final processing before reclamation
B. It replaces the constructor entirely
C. It initializes an object's fields
D. It converts an object into a subclass instance

Ans: A

Explanation: Finalization is conceptually linked to the end-of-life cleanup opportunity for an object.

Q209 | Mixed OAVS TGT Practice Set IDENTIFICATION

A superclass reference variable assigned a subclass object, followed by a call to an overridden method, will execute:

A. The subclass's version of the method
B. Neither version, causing an error
C. The superclass's version of the method
D. Both versions

Ans: A

Explanation: This is the standard behavior of dynamic method dispatch based on the object's actual runtime type.

Q210 | Mixed OAVS TGT Practice Set ASSERTION-REASON

Assertion (A): A well-designed class typically keeps its instance variables private and exposes public methods to interact with them. Reason (R): This approach supports both data hiding and encapsulation.

A. A is false, R is true
B. A is true, R is false
C. Both are false
D. Both A and R are true, and R correctly explains A

Ans: D

Explanation: Restricting direct field access while providing controlled public methods is the standard practice embodying data hiding and encapsulation together.